



Máster Universitario en Inteligencia Artificial

2025-2026

MLOPs

Lab 6

Pregunta 1

Si arrancamos el servidor con 127.0.0.1 (localhost), el contenedor se aísla y solo atiende peticiones que nazcan dentro de él mismo, ignorando cualquier conexión desde nuestro equipo. Por el contrario, asignar la IP comodín 0.0.0.0 ordena al contenedor que abra todas sus interfaces de red. Este paso es fundamental para que Docker pueda enrutar el tráfico de nuestra máquina (a través del puerto mapeado 8000) hacia la aplicación interna.

Pregunta 2

Dentro de un entorno de contenedores, el término localhost hace referencia exclusiva al sistema de red aislado del propio contenedor en el que se ejecuta el código, por lo que el frontend intentaría buscar la API dentro de sí mismo en lugar de salir a buscarla al otro servicio. Para resolver esta comunicación, Docker Compose crea una red interna privada donde actúa como un servidor DNS, registrando automáticamente el nombre de cada servicio definido (en este caso, backend) y vinculándolo con su dirección IP interna. De esta forma, el nombre del servicio se convierte en una dirección válida y localizable para todos los componentes que comparten esa misma red.

Pregunta 3

Docker genera una capa inmutable en caché por cada instrucción del Dockerfile, invalidando todas las capas posteriores si una se modifica. Si se copiara el código completo antes de instalar las dependencias, modificar una sola línea en main.py rompería la caché del código y forzaría a reinstalar todas las librerías desde cero, elevando el tiempo de construcción a varios minutos. Al instalar primero desde requirements.txt, las dependencias quedan cacheadas y, en el desarrollo diario, solo se recopia el código actualizado en cuestión de segundos, dejando las reconstrucciones largas únicamente para cuando se añade una dependencia nueva.

Pregunta 4

El principal riesgo de usar un archivo .env es que el token se almacena en texto plano en el disco local, por lo que si el archivo .gitignore falla por un error humano, el secreto se subirá al repositorio de código quedando expuesto públicamente. En cuanto a las alternativas, pasar el secreto como variable de entorno del shell implica exportarlo en la sesión del operador y se elegiría para entornos efímeros como pipelines de integración continua (CI). Docker Secrets cifra la contraseña en el plano de control y la monta en una ruta segura del contenedor, siendo la opción natural si el entorno de producción está orquestado con Docker Swarm. Finalmente, los servicios externos como AWS Secrets Manager o Vault hacen que la aplicación recupere la clave en tiempo de ejecución a través de una API con autenticación, y son la alternativa ideal para infraestructuras de producción cloud o arquitecturas multi-entorno.

Escenario 1

Al ser una carga casi inmediata, el tiempo que transcurre entre que se inicia el contenedor y el modelo está listo en memoria es imperceptible. En este caso tan optimista, un arranque secuencial clásico (un `depends_on` simple) no suele dar problemas, ya que es poco probable que el frontend colapse.

Escenario 2

Aquí la latencia de la red es el factor crítico. El contenedor del backend se enciende en milisegundos, momento en el que un `depends_on` simple daría luz verde al frontend para arrancar. Sin embargo, el backend estará varios segundos bloqueado descargando el archivo. Si el frontend intenta enviar datos durante esa ventana de tiempo, la conexión fallará.

Escenario 3

Es el escenario más inestable. El backend arranca, pero tarda decenas de segundos en preparar el modelo para inferencia. Levantar la interfaz web sin verificar el estado real de la memoria provocará errores 500 en cascada en cuanto el usuario intente usar la aplicación prematuramente.

Pseudocódigo

1. LÓGICA DEL BACKEND (FastAPI)

Evento de inicio (Startup):

```
Disparar Hilo_Secundario -> Descargar_Modelo_HF()  
Estado_Global = "Cargando_Modelo"
```

Al finalizar Hilo_Secundario:

```
Cargar_Pesos_En_Memoria()  
Estado_Global = "Modelo_Listo"
```

Endpoint GET /health:

```
SI Estado_Global == "Modelo_Listo":  
    Devolver HTTP 200 (OK - Saludable)
```

SINO:

```
    Devolver HTTP 503 (Servicio No Disponible)
```

2. LÓGICA DE ORQUESTACIÓN (docker-compose.yml)

Servicio Backend:

Healthcheck: Hacer ping a GET /health cada 10 segundos.

Servicio Frontend:

Arranque: Esperar a que Servicio Backend pase la prueba (condición: `service_healthy`).